



Федеральное государственное автономное образовательное

учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное
обеспечение

Дисциплина «Вычислительная математика»

Отчёт по лабораторной работе №3

Вариант - 6

Выполнил

Линейский Аким Евгеньевич

P3215

Проверил

Наумова Надежда Александровна

Санкт - Петербург 2026

Содержание

1. Содержание.....	2
2. Цель	3
3. Описание метода.....	Ошибка! Закладка не определена.
4. Исходный код программы.....	Ошибка! Закладка не определена.
5. Результат работы.....	8
6. Вывод	11

Цель

Найти приближенное значение определенного интеграла с требуемой точностью различными численными методами, выполнить программную реализацию методов на языке Python.

Порядок выполнения работы

1. Программная реализация задачи
 - 1.1. Блок-схемы численных методов
 - 1.1. Реализация численных методов и функций
2. Вычислительная реализация задачи
 - 2.1. Точное вычисление интеграла
 - 2.2. Вычисление по формуле Ньютона-Котеса
 - 2.3. Вычисление по формуле средних прямоугольников
 - 2.4. Вычисление по формуле трапеции
 - 2.5. Вычисление по формуле Симпсона
 - 2.6. Сравнение полученных результатов

Блок-схемы численных методов

Методы прямоугольников

ф

Метод трапеций

ф

Метод Симпсона

ф

Исходный код вычислительных методов

```
def left_rectangles(fun, a, b, n):
    result = 0
    h = (b-a) / n
    for i in range(n):
        result += h * fun(a+h*i)
    return result

def right_rectangles(fun, a, b, n):
    result = 0
    h = (b-a) / n
    for i in range(n):
        result += h * fun(a+h*(i+1))
    return result

def center_rectangles(fun, a, b, n):
    result = 0
    h = (b-a) / n
    for i in range(n):
        result += h * fun(a+h/2 + h*i)
    return result

def trapezoid(fun, a, b, n): # If n<3 ?
    if n < 1:
        return 0

    result = 0
    h = (b-a) / n
    for i in range(1, n):
        result += fun(a+h*i)
    result += (fun(a) + fun(b))/2

    return result * h

def simpson(fun, a, b, n):
    if n % 2 != 0:
        n = n if n % 2 == 0 else n + 1

    result = 0
    h = (b-a) / n
    for i in range(n+1):
        y_i = fun(a+h*i)
        if i%2==0:
            result += 2 * y_i
        else:
            result += 4 * y_i

    return result* h/3
```

```

def runge_rule(I1, I0, k, e):
    R = abs(I1 - I0) / (2**k - 1)
    return R < e

def calucale_integral(fun, method, k, a, b, e, logs=False, max_iter=100):
    """
    Вычисление интеграла с заданной точностью
    fun - интегрируемая функция
    method - метод интегрирования (функция)
    k - порядок точности метода
    a, b - пределы интегрирования
    e - требуемая точность
    """

    # TODO: Обработка разрыва 2 рода
    # TODO: если выйти из области определения - не решается

    n = 4
    i = 0
    integrals = []

    while i < max_iter:
        try:
            current_integral = method(fun, a, b, n)
            integrals.append(current_integral)
        except Exception as ex:
            print(f"Ошибка при вызове метода: {ex}")
            print(f"method={method}, fun={fun}, a={a}, b={b}, n={n}")
            raise

        if logs:
            print(f"[{i}]: n={n}, integral={current_integral}")

        if i >= 1:
            if runge_rule(current_integral, integrals[-2], k, e):
                if logs:
                    print(f"Достигнута точность на итерации {i}, n={n}")
                break

        n *= 2
        i += 1

    return integrals[-1]
# Методы решения уравнений
@staticmethod
def solve_bisection(f, a, b, eps, max_iter=100):
    history = []
    if f(a) * f(b) > 0:
        raise ValueError("На концах интервала функция должна иметь разные
знаки")

```

```

for i in range(max_iter):
    c = (a + b) / 2
    fc = f(c)
    err = (b - a) / 2

    history.append({
        "iter": i+1,
        "a": a,
        "b": b,
        "x": c,
        "f(x)": fc,
        "error": err
    })

    if err < eps or abs(fc) < eps:
        break

    if f(a) * fc < 0:
        b = c
    else:
        a = c

return c, history

@staticmethod
def solve_secant(f, x0, x1, eps, max_iter=100):
    history = []
    x_prev, x_curr = x0, x1

    for i in range(max_iter):
        f_prev = f(x_prev)
        f_curr = f(x_curr)

        if abs(f_curr - f_prev) < 1e-15:
            break

        x_next = x_curr - f_curr * (x_curr - x_prev) / (f_curr - f_prev)
        err = abs(x_next - x_curr)

        history.append({
            "iter": i+1,
            "x_prev": x_prev,
            "x_curr": x_curr,
            "f(x_prev)": f_prev,
            "f(x_curr)": f_curr,
            "x_next": x_next,
            "error": err
        })

        if err < eps or abs(f(x_next)) < eps:
            x_curr = x_next
            break

```

```

        x_prev, x_curr = x_curr, x_next

    return x_curr, history

@staticmethod
def solve_simple_iteration(phi, x0, eps, max_iter=100):
    history = []
    x_curr = x0

    for i in range(max_iter):
        x_next = phi(x_curr)
        err = abs(x_next - x_curr)

        history.append({
            "iter": i+1,
            "x": x_curr,
            "phi(x)": x_next,
            "error": err
        })

        if err < eps:
            x_curr = x_next
            break

    x_curr = x_next

    return x_curr, history

# Метод решения систем
@staticmethod
def solve_newton_system(F, J, x0, y0, eps, max_iter=100):
    history = []
    x, y = x0, y0

    for i in range(max_iter):
        f1, f2 = F(x, y)
        jac = J(x, y)

        try:
            det = jac[0,0] * jac[1,1] - jac[0,1] * jac[1,0]
            dx = (jac[1,1] * f1 - jac[0,1] * f2) / det
            dy = (-jac[1,0] * f1 + jac[0,0] * f2) / det
        except:
            break

        x_new, y_new = x - dx, y - dy
        err = max(abs(x_new - x), abs(y_new - y))

        history.append({
            "iter": i+1,
            "x": x,
            "y": y,
            "F(x,y)": f1, f2,
            "J(x,y)": jac,
            "error": err
        })

        if err < eps:
            x, y = x_new, y_new
            break

    x, y = x_new, y_new

    return x, y, history

```

```

        "y": y,
        "f1(x,y)": f1,
        "f2(x,y)": f2,
        "error": err
    })

    if err < eps:
        x, y = x_new, y_new
        break

    x, y = x_new, y_new

return (x, y), history

```

Код доступен по ссылке на удаленный репозиторий GitHub:

[Код программы](#)

Вычислительная реализация задачи

$$f(\bar{x}_i) = 3x_i^3 + 5x_i^2 + 3x_i - 6$$

Точное вычисление

$$\int_1^2 (3x^3 + 5x^2 + 3x - 6)dx$$

$$F(x) = \int (3x^3 + 5x^2 + 3x - 6)dx = \frac{3x^4}{4} + \frac{5x^3}{3} + \frac{3x^2}{2} - 6x$$

$$I = F(2) - F(1)$$

$$F(2) = \frac{3 * 2^4}{4} + \frac{5 * 2^3}{3} + \frac{3 * 2^2}{2} - 6 * 2 = \frac{48}{4} + \frac{40}{3} + \frac{12}{2} - 12 = \frac{58}{3}$$

$$F(1) = \frac{3 * 1^4}{4} + \frac{5 * 1^3}{3} + \frac{3 * 1^2}{2} - 6 * 1 = \frac{3}{4} + \frac{5}{3} + \frac{3}{2} - 6 = -\frac{25}{12}$$

$$I = \frac{58}{3} - \left(-\frac{25}{12}\right) = \frac{232}{12} + \frac{25}{12} = \frac{257}{12} \approx \mathbf{21,4166666667}$$

Формула Ньютона-Котеса

$$h = \frac{2-1}{6} = \frac{1}{6}, n = 6$$

$$I \approx \frac{b-a}{840} (41f_0 + 216f_1 + 27f_2 + 272f_3 + 27f_4 + 216f_5 + 41f_6)$$

$$f(1) = 5$$

$$f(1.1667) \approx 9.0694$$

$$f(1.3333) \approx 14.000$$

$$f(1.5) \approx 19.8750$$

$$f(1.6667) \approx 26.7778$$

$$f(1.8333) \approx 34.7917$$

$$f(2) = 44$$

$$I \approx \frac{1}{840} * 17990 \approx \mathbf{21.41667}$$

Формула средних прямоугольников

$$h = \frac{2-1}{10} = 0.1, \quad \bar{x}_i = x_i + \frac{h}{2}$$

i	$\bar{x}_i$	$f(\bar{x}_i)$
1	1.05	6.1354
2	1.15	8.6251
3	1.25	11.4219
4	1.35	14.5436
5	1.45	18.0084
6	1.55	21.8341
7	1.65	26.0389

8	1.75	30.6406
9	1.85	35.6574
10	1.95	44.1071

$$I \approx h * \sum_1^{10} f(\bar{x}_i) = 0.1 * 217.0125 = \mathbf{21.70125}$$

Формула трапеций

$$h = \frac{2 - 1}{10} = 0.1$$

i	x_i	$f(x_i)$
0	1	5
1	1.1	7.3430
2	1.2	9.9840
3	1.3	12.9410
4	1.4	16.2320
5	1.5	19.8750
6	1.6	23.8880
7	1.7	28.2890
8	1.8	33.0960
9	1.9	38.3270
10	2	44

$$I \approx h * \left(\frac{f(x_0) + f(x_{10})}{2} + \sum_{i=1}^9 f(x_i) \right)$$

$$I \approx 0.1 * \left(\frac{5 + 44}{2} + 7.343 + 10.008 + 13.019 + 16.400 + 20.175 + 24.368 + 29.003 + 34.104 + 39.695 \right) = \mathbf{21.4475}$$

Формула Симпсона

$$I \approx \frac{h}{3} * \left(f(x_0) + 4 * \sum_0^4 f(x_{2k+1})) + 4 * \sum_1^4 f(x_{2k})) + f(x_{10}) \right)$$

$$I \approx \frac{0.1}{3} * (5 + 4 * 109.235 + 2 * 84.88 + 44) = \mathbf{21.41667}$$

Сравнение результатов

Метод	Результат	Относительная погрешность
Точное значение	21.4166667	0
Ньютона-Котеса	21.41667	0.000000155642023
Ср. прямоугольников	21.70125	0.013287937743191
Трапеций	21.4475	0.001439688715953
Симпсона	21.41667	0.000000155642023

Результат работы

```
Вычисление интеграла с параметрами:
fun: function_2
method_func: center_rectangles
k: 2
a: -1.0, b: 1.0, e: 0.001
[0]: n=4, integral=0.625
[1]: n=8, integral=0.65625
[2]: n=16, integral=0.6640625
[3]: n=32, integral=0.666015625
Достигнута точность на итерации 3, n=32
Найденный приближенный интеграл: 0.666015625

Выберите опцию:
0 - выход
1 - задать пределы
2 - задать точность
3 - выбрать функцию
4 - выбрать метод
5 - вычислить интеграл
Ваш выбор: |
```

Вывод

Прodelав данную лабораторную работу №3 я изучил способы вычисления определенного интеграла различными методами, реализовал численные методы на языке Python.

